DEEP LEARNING

REPORTE PROYECTO 1 AUTOENCODERS

Alumnos: Daniela de la Torre Gallo
Aissa Berenice González Fosado
Clara Paola Aguilar Casillas

Docente: Ivan Reyes Amezcua

ITESO, Universidad Jesuita de Guadalajara

Index

- Introduction
- Objectives
- Dataset
- Data Preparation
    - Normalization
    - Data Split
- Modeling
    - Deep (Dense) Autoencoder
    - Convolutional Autoencoder
    - Denoising Autoencoder
    - Variational Autoencoder + New Image Generation
- Conclusions

Introduction

In this project, we worked with different types of **Autoencoders**, which are Deep Learning models that learn compressed representations of images and then reconstruct them.

Our main goal was to compare different architectures and understand how compression affects reconstruction quality and image generation.

Objectives

- Understand the fundamental principles of autoencoders and their application in generative deep learning.
- Implement a basic autoencoder and variational autoencoders for a specific task, such as dimensionality reduction, denoising, or image generation.
- Analyze the performance and characteristics of the representations learned by the autoencoders.

Dataset

We constructed a custom dataset of 1,000 glasses images using web scraping to collect the required samples. To ensure consistency and improve training performance, all images share the same orientation and have a white background.

This standardization helps reduce unnecessary variability, which is especially important given the relatively small dataset size used for training the autoencoder. We did the rest of the preparation on the notebook.

Dataset Preparation

All images were resized to 32×32 pixels with three RGB channels to ensure a uniform input shape for the model. After resizing, we verified that each image maintained a consistent format and final shape of (32, 32, 3). The images were then converted into NumPy arrays so they could be processed efficiently by the neural network.

Maintaining consistent dimensions is essential, as neural networks require fixed input sizes to function properly. The final dataset contains the same 1,000 images, all standardized to the same resolution and format.

Total images: 1000

All images resized to: (32, 32)

Data Preparation

Normalization

We are converting the image arrays to `float32` and normalizing them by dividing by 255, since RGB color images have pixel values ranging from 0 to 255. This scales the data to the [0, 1] range, which improves numerical stability and helps the autoencoder train more efficiently.

Data Split

The resized images are converted to a NumPy array and split into 80% training, 10% validation, and 10% test sets with a fixed random_state for reproducibility.

For our experiments, the training set learns from these samples, the validation set tunes and monitors the model, and the test set evaluates final performance and is held for the final inspection.

Model 1

Deep (Fully Connected) Autoencoder

The model first flattens the image into a 1-D vector, progressively reduces dimensionality through stacked Dense layers, compresses the representation into an 8-dimensional latent vector, and then mirrors the architecture to reconstruct the original image.

This was our simplest AE yet we had to keep making changes to it, since we got with millions of parameters when we started, this was fixed reducing the dimensionality of our first layer and our latent space.

Architecture

Encoder

Layer	Type	Output Shape	Description
Input	Input	(32, 32, 3)	Original image
Flatten	Flatten	(3072)	Converts image into a 1D vector (32×32×3)
Dense	128 neurons (ReLU)	(128)	First compression stage
Dense	64 neurons (ReLU)	(64)	Further compression
Dense	16 neurons (ReLU)	(16)	High-level compact features

Latent Space

Layer	Type	Output Shape	Description
Latent Space	8 neurons	(8)	Final compressed representation (bottleneck)

Decoder

Layer	Type	Output Shape	Description
Dense	16 neurons (ReLU)	(16)	Start expanding from latent space
Dense	64 neurons (ReLU)	(64)	Continue expansion
Dense	128 neurons (ReLU)	(128)	Recover higher dimensional features
Dense	3072 neurons (Sigmoid)	(3072)	Reconstruct all pixel values (0–1)
Reshape	Reshape	(32, 32, 3)	Convert vector back to image shape

Model: "DeepAutoencoder"

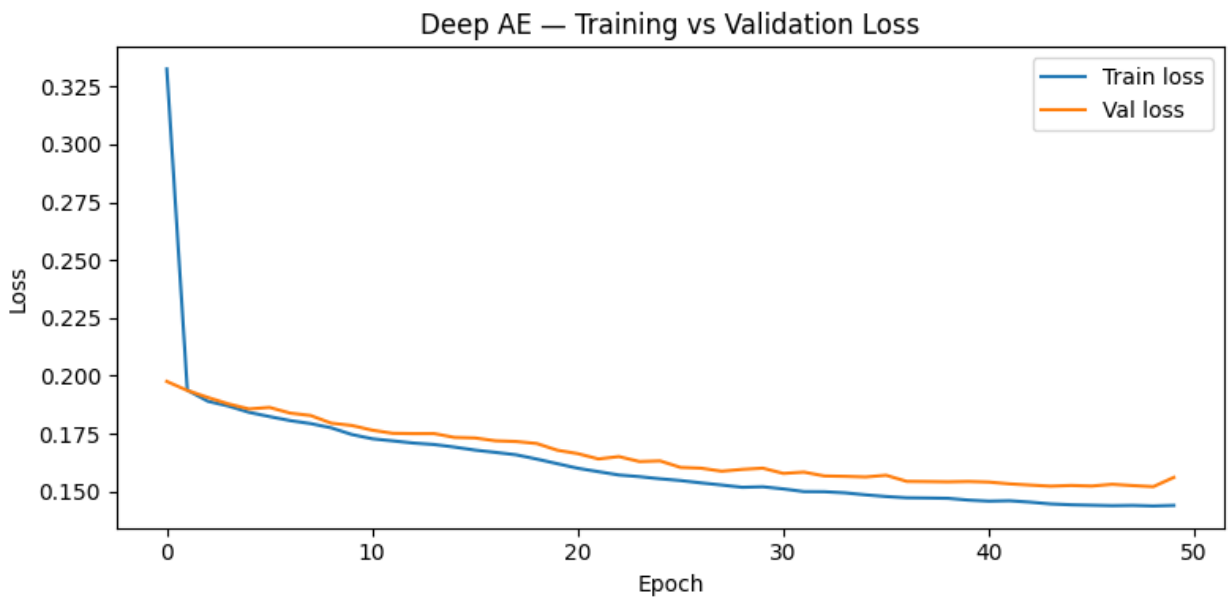
Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 32, 32, 3)	0
flatten_1 (Flatten)	(None, 3072)	0
dense_7 (Dense)	(None, 128)	393,344
dense_8 (Dense)	(None, 64)	8,256
dense_9 (Dense)	(None, 16)	1,040
latent_space (Dense)	(None, 8)	136
dense_10 (Dense)	(None, 16)	144
dense_11 (Dense)	(None, 64)	1,088
dense_12 (Dense)	(None, 128)	8,320
dense_13 (Dense)	(None, 3072)	396,288
reshape_1 (Reshape)	(None, 32, 32, 3)	0

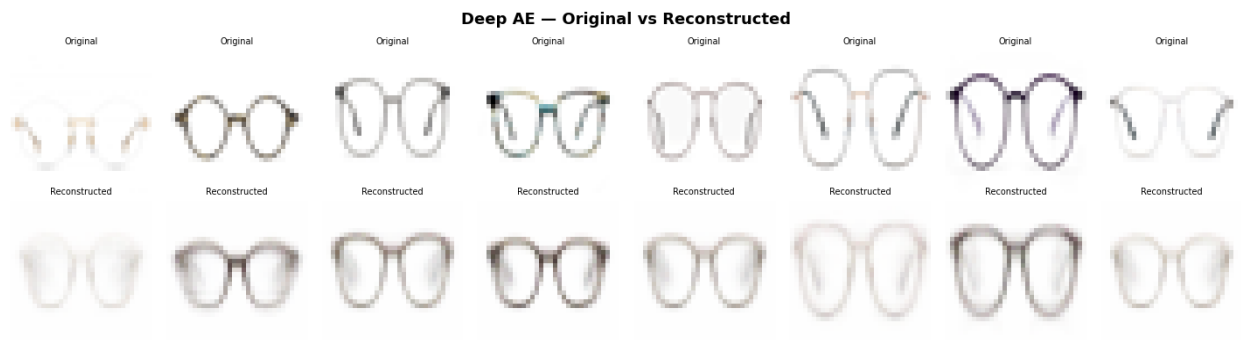
Total params: 808,616 (3.08 MB)

Trainable params: 808,616 (3.08 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 50/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.1440 - val_loss: 0.1561
```





Model 2

Convolutional Autoencoder (CAE)

For this project, a convolutional autoencoder (CAE) is a more suitable choice than a fully connected architecture. Dense layers process every pixel independently and do not account for the spatial relationships that define images. As a result, they often require a large number of parameters to learn even simple visual patterns.

We saw a huge jump from 808,616 parameters (That were already very hard to get this low), down to 75,651 parameters, making this Convolutional autoencoder much more efficient.

Convolutional layers are specifically designed for image data: they share weights across spatial locations and learn local patterns such as edges, textures, and shapes. This not only reduces the total number of trainable parameters but also allows the model to better preserve structural information during reconstruction. Consequently, a CAE

typically produces sharper and more visually coherent outputs while remaining computationally efficient for images of this size and we noticed it on our graphs below.

Architecture

Encoder

Layer	Output Shape	Description
Conv2D 32 × (3×3), ReLU, same	32×32×32	Detects low-level features (edges, colours); same padding keeps spatial dimensions
MaxPool2D 2×2	16×16×32	Halves spatial resolution and keeps strongest activations
Conv2D 64 × (3×3), ReLU, same	16×16×64	Learns higher-level patterns
MaxPool2D 2×2	8×8×64	Further downsampling leading to the latent space

Decoder

Layer	Output Shape	Description
Conv2DTranspose 64 × (3×3), ReLU, same	8×8×64	Learnable upsampling
UpSampling2D 2×2	16×16×64	Doubles spatial resolution
Conv2DTranspose 32 × (3×3), ReLU, same	16×16×32	Refines reconstructed features
UpSampling2D 2×2	32×32×32	Restores original resolution
Conv2DTranspose 3 × (3×3), Sigmoid, same	32×32×3	Final RGB output in range [0,1]

Model: "Conv_AE"

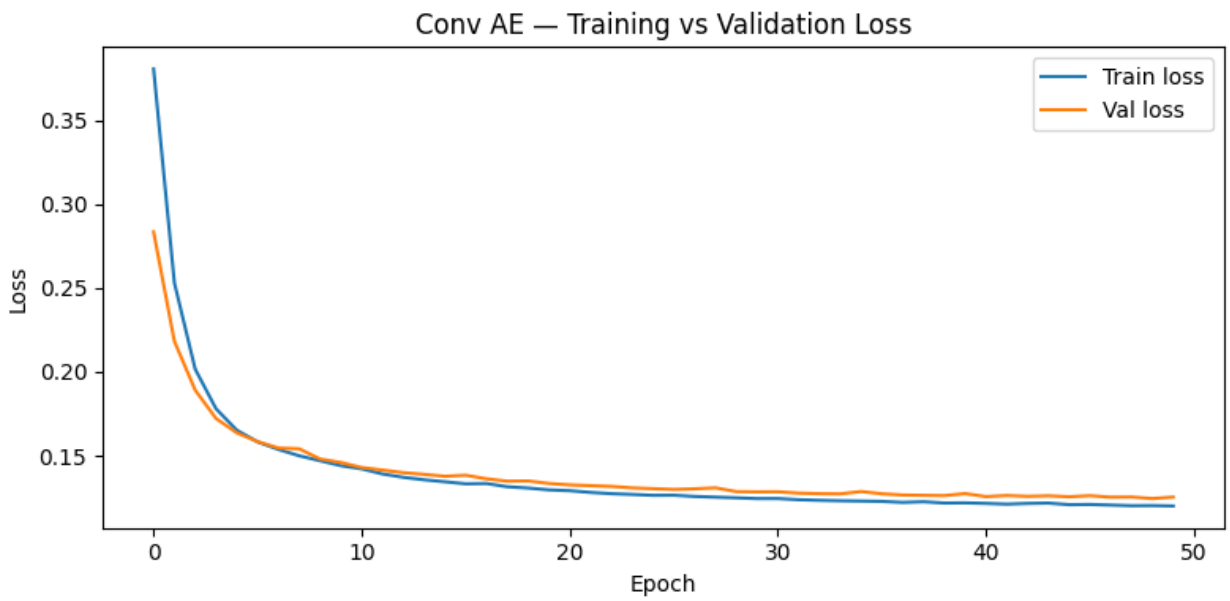
Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 32, 32, 3)	0
conv2d_8 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d_8 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_9 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_9 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_transpose_12 (Conv2DTranspose)	(None, 8, 8, 64)	36,928
up_sampling2d_8 (UpSampling2D)	(None, 16, 16, 64)	0
conv2d_transpose_13 (Conv2DTranspose)	(None, 16, 16, 32)	18,464
up_sampling2d_9 (UpSampling2D)	(None, 32, 32, 32)	0
conv2d_transpose_14 (Conv2DTranspose)	(None, 32, 32, 3)	867

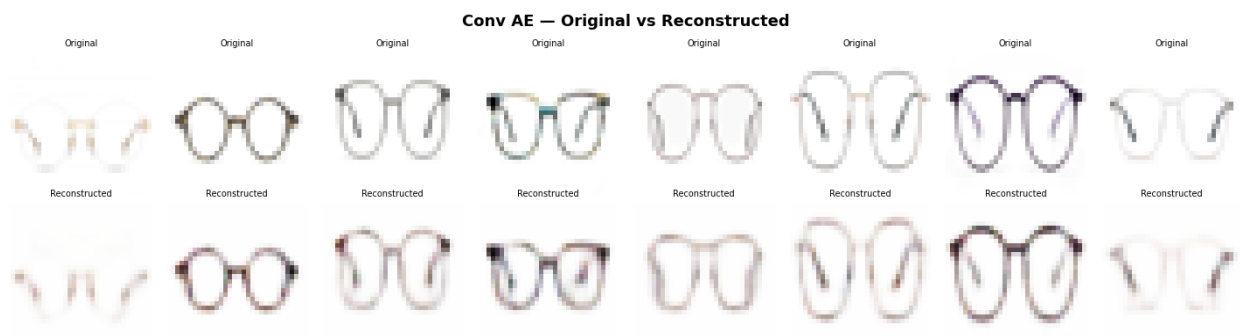
Total params: 75,651 (295.51 KB)

Trainable params: 75,651 (295.51 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 50/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step - loss: 0.1201 - val_loss: 0.1254
```





Model 3

Denoising Autoencoder (DAE)

On our project, a denoising variant of the convolutional autoencoder is a natural extension. A denoising autoencoder is trained by **corrupting the input** with random noise and asking the network to reconstruct the **clean** original. This forces the encoder to learn truly meaningful structure rather than an identity mapping, acting as a regulariser and improving feature quality.

Noise strategy

We corrupted the input with Gaussian noise with standard deviation stddev = 0.15. After adding noise, values are clipped to the [0,1] range so the network always sees valid pixel intensities.The chosen noise level is large enough to produce clearly corrupted

examples while preserving the underlying structure the model must learn to recover; this encourages the encoder to extract robust, generalizable features

Architecture

Identical to the Convolutional AE — the only change is that noisy images are passed as input while clean images remain the target. This demonstrates how the same architecture behaves differently depending on the training objective.

Layer	Dimensions	Description
Input	32×32×3	Noisy RGB image supplied to the network during training
Encoder	32×32×32 → MaxPool → 16×16×32 → 16×16×64 → MaxPool → 8×8×64	Convolutional downsampling that learns local features and compresses spatial information into a compact feature map
Latent / Map	8×8×64	Encoded spatial feature map (latent representation in convolutional autoencoder)
Decoder	8×8×64 → ConvTranspose/Up → 16×16×64 → ConvTranspose/Up → 32×32×32 → ConvTranspose (3, Sigmoid) → 32×32×3	Symmetric upsampling and refinement to reconstruct the RGB image in range [0,1]
Output	32×32×3	Clean ground-truth image used as reconstruction target

Model: "Denoising_AE"

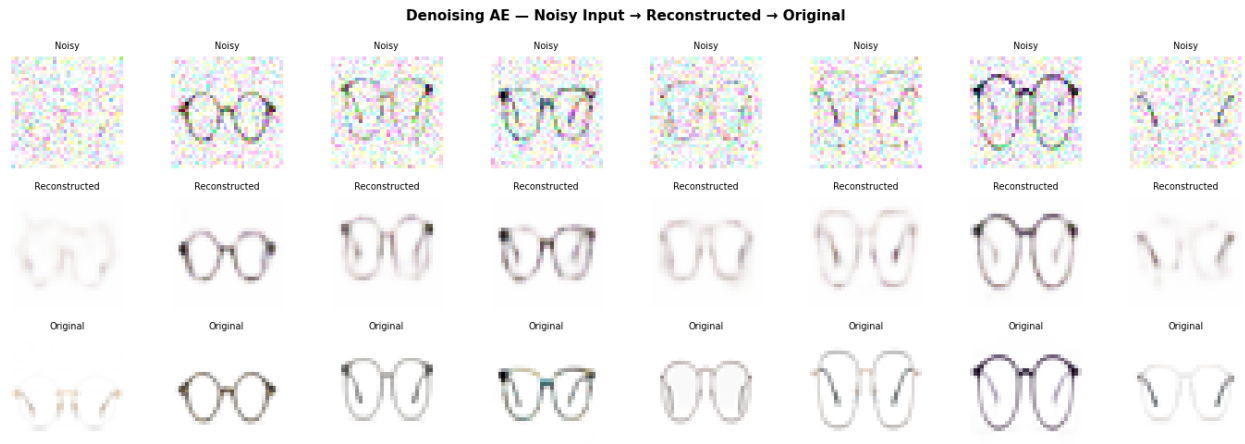
Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 32, 32, 3)	0
conv2d_10 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d_10 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_11 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_11 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_transpose_15 (Conv2DTranspose)	(None, 8, 8, 64)	36,928
up_sampling2d_10 (UpSampling2D)	(None, 16, 16, 64)	0
conv2d_transpose_16 (Conv2DTranspose)	(None, 16, 16, 32)	18,464
up_sampling2d_11 (UpSampling2D)	(None, 32, 32, 32)	0
conv2d_transpose_17 (Conv2DTranspose)	(None, 32, 32, 3)	867

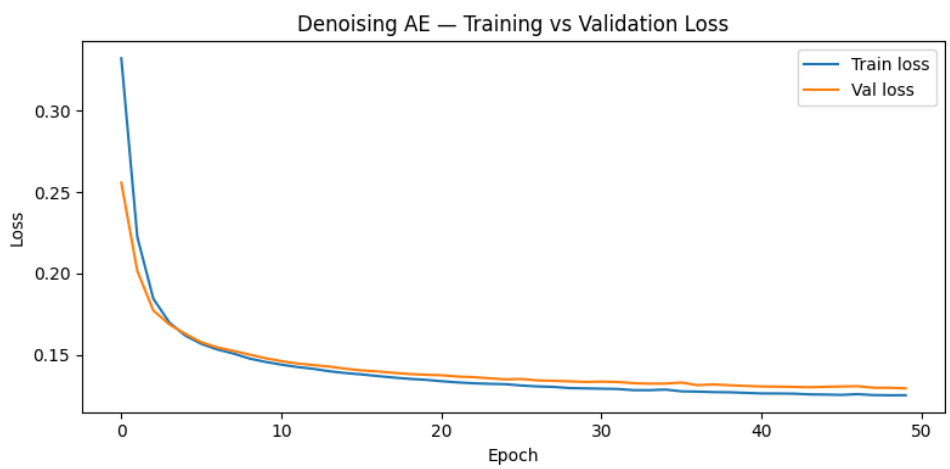
Total params: 75,651 (295.51 KB)

Trainable params: 75,651 (295.51 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 50/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.1252 - val_loss: 0.1294
```





Model4

Variational Autoencoder (VAE)

A Variational Autoencoder is a generative neural network that learns to compress images into a probabilistic latent space and then reconstruct them. Instead of mapping each image to a single fixed point, it learns a mean and variance that define a distribution. By sampling from this distribution and decoding it, the model can generate new images that resemble the training data. The loss function combines reconstruction error (to keep outputs realistic) and KL divergence (to organize the latent space into a smooth normal distribution).

Since our project consists of generation new images of the glasses too, not just reconstructing the images. The structured latent space allows us to sample random vectors and create new designs that follow the patterns learned during training.

This worked well for the glasses dataset because glasses share consistent shapes and symmetry, which convolutional layers capture effectively from what we saw on earlier experiments. The 16-dimensional latent space was enough to represent key variations (frame shape, thickness, style) without overfitting, and the KL regularization ensured that sampling produced realistic new glasses instead of noise.

Architecture

Layer	Type	Output Shape	Description
Input	Input	(32, 32, 3)	Glasses image
Conv1	Conv2D (32 filters, stride 2)	(16, 16, 32)	Extracts low-level features like edges and basic shapes
Conv2	Conv2D (64 filters, stride 2)	(8, 8, 64)	Learns deeper visual patterns
Flatten	Flatten	(4096)	Converts feature map into a vector
Dense	Dense (128)	(128)	High-level representation
z_mean	Dense (16)	(16)	Learns the mean of the latent distribution
z_log_var	Dense (16)	(16)	Learns the log variance of the latent distribution
Sampling	Custom Layer	(16)	Samples latent vector **z** using reparameterization trick

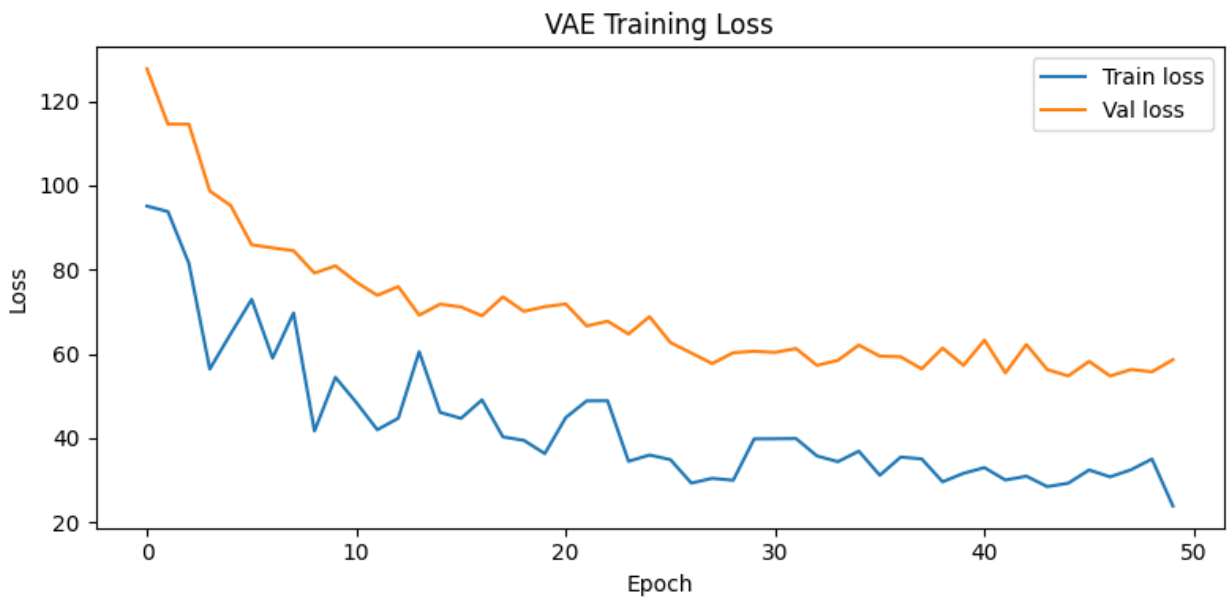
Latent Space

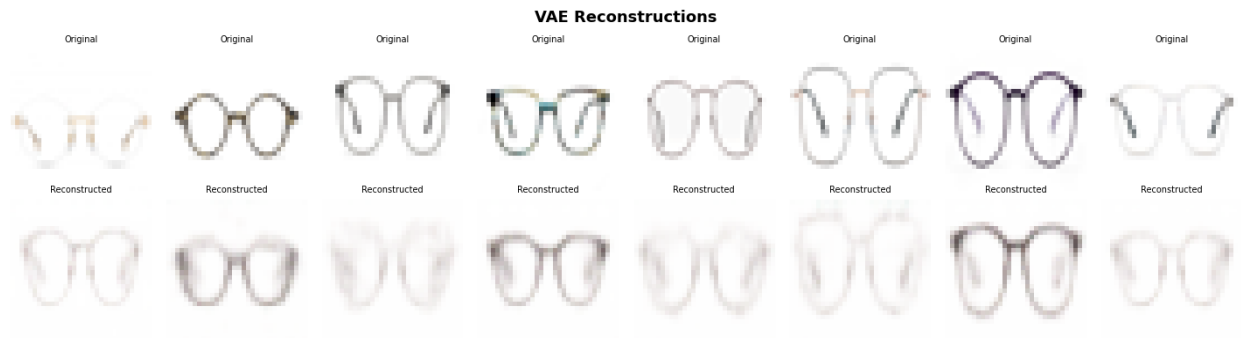
Component	Dimension	Purpose
Latent Vector (z)	16	Compressed probabilistic representation of the glasses

Decoder

Layer	Type	Output Shape	Description
Input	Input	(16)	Sampled latent vector
Dense	Dense (4096)	(4096)	Expands latent representation
Reshape	Reshape	(8, 8, 64)	Converts vector back to feature map
Deconv1	Conv2DTranspose (64, stride 2)	(16, 16, 64)	Upsampling
Deconv2	Conv2DTranspose (32, stride 2)	(32, 32, 32)	Further upsampling
Output	Conv2D (3 filters, sigmoid)	(32, 32, 3)	Final generated RGB image in [0,1]

```
Epoch 50/50
25/25 - 0s - 15ms/step - kl_loss: 4.4473 - loss: 23.8775 - recon_loss: 19.4302 - val_kl_loss: 7.1847 - val_loss: 58.6012 - val_recon_loss: 51.4165
```





Generating a new image

Our new image was visibly very close to glasses most of the time the generation was made, we have saved it into the repository and an example is shown below.

Conclusions

This project demonstrated how different autoencoder architectures behave on a small, standardized glasses dataset (32×32 resolution). The fully connected autoencoder worked as a basic compression–reconstruction model but struggled to preserve spatial structure. The convolutional autoencoder significantly improved reconstruction quality while using far fewer parameters, showing that convolutional layers are more suitable for image data.

The denoising autoencoder further improved feature robustness by learning to recover clean images from noisy inputs. Finally, the variational autoencoder successfully learned a structured latent space that allowed the generation of new glasses images that were visually consistent with the dataset, the metrics were lower but the recreation was kept very accurate.

Overall, the experiments confirmed that architecture choice strongly impacts reconstruction quality, efficiency, and generative capability, the patterns on our dataset were very recognizable on the architectures and models we chose.